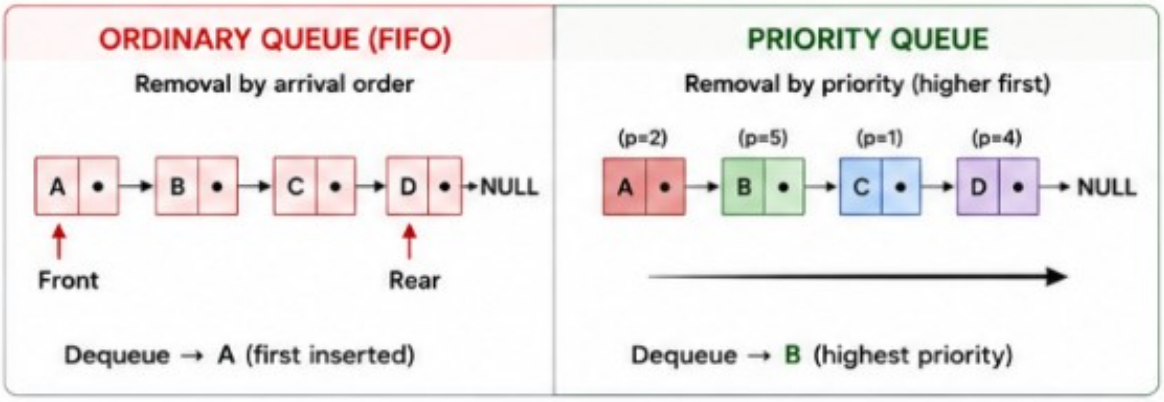
Priority Queue

So far, we have learnt the data structures list, stack, circular list and queue. We have observed that these are all abstract data types supporting a well-defined interface. We have also observed that each of these abstract data types could have different implementations. These data structures are found to be useful in different applications based on their interfaces.

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.



A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority. In Unix, the latter concept is followed. You may know that there is a command called nice using which you can reduce the priority of your process – you can be nice to others!

In our examples, we will assume that higher the numeric value, higher the priority.

What is the rationale behind this? Let us consider a couple of examples.

The patients wait in a queue to consult a doctor. But what if a patient becomes critically ill? Should not that patient get immediate attention?

Several processes could be running on an operating system. A few of them may have to respond in real time. Should not these get preference over others?

Our temples have an unusual way of implementing the priority queue. If you ready to pay more, you will get more priority!

Interface of the priority queue:

The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full

Implementation of priority queue:

A priority queue is a collection of elements in which every element has an associated priority, and deletion returns an element having the appropriate extreme priority.

A priority is an abstract data type. As we know, an **Abstract Data Type (ADT)** specifies **what operations are supported and what they mean**, without specifying how those operations are implemented.

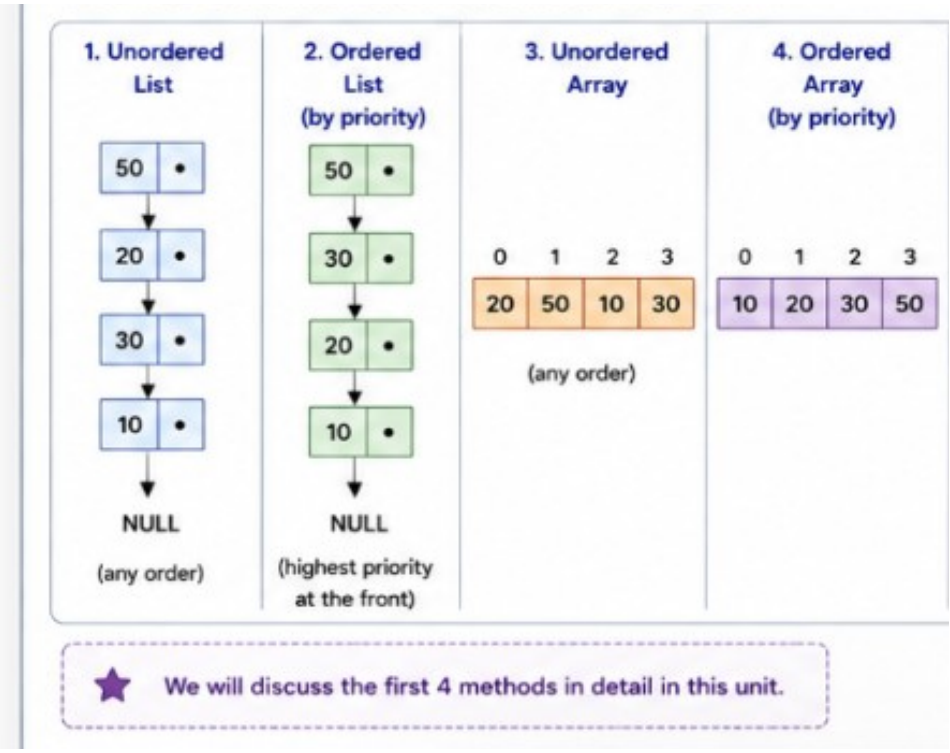
A priority queue can be implemented using any suitable underlying data structure. We will discuss the first 4 of the 5 listed below. The last one shall be discussed later in this course.

Different ways in which we may implement the priority queue:

1. Unodered list
2. Ordered list : ordered based on priority
3. Unordered array
4. Ordered array : ordered based on priority
5. Heap tree

Please note that these are not the only ways of implementing the ADT priority queue.

We will discuss the first 4 ways of implementing the priority queue in detail in the next few sections.



Applications of priority queue :

- CPU Scheduling of processes
- Hospital Emergency Services
- Printer Scheduling
- Network Packet Scheduling
- Dijkstra's Shortest Path Algorithm
- Prim's Minimum Spanning Tree Algorithm
- Event-Driven Simulation
- Huffman Coding

Comparison of queue and priority queue:

Priority Queue vs Ordinary Queue

Feature	**Ordinary Queue**	**Priority Queue**
Ordering principle	FIFO	Priority
Important attribute	Arrival time	Priority
Insert	Usually at rear	Huffman Coding
Delete	From front	Highest/lowest priority
Example	Printer jobs	CPU scheduling
Typical implementation	Array / linked list	Heap / ordered or unordered list

Comparison of various implementations:

We will discuss these again after all 4 ways of implementation are completed.

Implementation	**Insert**	**Delete priority item**
Unordered list	A few operations	Number of operations Proportional to number of elements
Ordered list	Number of operations Proportional to number of elements	A few operations
Unordered Array	A few operations	Number of operations Proportional to number of elements
Ordered Array	Number of operations Proportional to number of elements	A few operations
Binary heap	Number of operations proportional to Log to base 2 of number of elements	Number of operations proportional to Log to base 2 of number of elements

Implementation:

We shall discuss in this section whatever is common to all the four implementations. Implementation specific to a particular selected data structure shall be discussed separately.

Job : We have called each component of the priority queue as a job – a term commonly used in operating systems. The job structure has a unique number to identify the job and a priority to indicate how it should be handled. We are assuming that higher this number, higher the priority.

Each new job is given the job number in an increasing order. This job number will help us make out in which order the jobs have been services.

```
struct job
{
        int job_no_;
        int priority_;
};
typedef struct job job_t;
```

This ADT job supports setting a job, displaying the job and return the priority of a job when queried.

```
void set_job(job_t* ptr_job, int job_no, int priority);
void disp_job(job_t* ptr_job);
int get_priority(job_t* ptr_job);
```

The following is the implementation of this interface. The code is self explanatory.

```
void set_job(job_t* ptr_job, int job_no, int priority)
{
        ptr_job->job_no_ = job_no;
        ptr_job->priority_ = priority;
```

```
}


void disp_job(job_t* ptr_job)
{
        printf("job # : %d priority : %d\n",
                ptr_job->job_no_, ptr_job->priority_);
}


int get_priority(job_t* ptr_job)
{
        return ptr_job->priority_;
}
```

Client :

The client code is menu driven. It reads an option before a loop is entered and an option again at the end of the loop. This is a characteristic of menu driven programs.

We support options for enque, deque and exiting the loop.

enque: This code takes the priority as input, increments the job number and calls enque of the underlying data structure. If the priority queue is full, the program is exited.

```
case 1 :        scanf("%d", &priority);
                set_job(&jobx, job_no++, priority);
                printf("Adding : ");
                disp_job(&jobx);
                enque(&q, &jobx);
                break;
```

deque: This code gets the job with the highest priority from the priority queue and then displays its contents. If the queue is empty, the program is exited.

```
case 2 :        jobx = deque(&q);
                printf("removing : ");
                disp_job(&jobx);
                break;
```

The job and the client will not change when we select different implementations of the priority queue.